NEXUS PRESENTATION & JUDGE DEFENSE

NEXUS

Presentation Preparation Guide & Technical Defense

A comprehensive, presenter-ready handbook designed to empower team members to confidently articulate, demonstrate, and defend the Nexus Autonomous Software Engineering Platform before hackathon technical judges.

EVALUATION EVENT
Hackathon Round 2: Implementation & Demo

TARGET AUDIENCE
Technical Evaluation Committee & Mentors

CORE DEMONSTRATION
Autonomous Self-Correction & Playwright Testing

IMPLEMENTATION STATUS
100% E2E Verified on Windows Host

1. Project Overview & Elevator Pitches

Nexus is an autonomous software engineering platform that acts as a specialized six-agent engineering department. Given a high-level software feature request in natural language, Nexus discovers requirements, blueprints the architecture, writes code in an isolated Git workspace, tests it in a real browser across desktop and mobile breakpoints, diagnoses visual defects, self-corrects the code, audits the diff, and delivers a verified GitHub Pull Request.

> **15-SECOND PITCH (RAPID OPENING)**
> "Nexus is an autonomous software engineering team powered by six specialized AI agents. You give it a feature request, and through isolated workspaces, human approval gates, real Playwright browser testing, and autonomous self-correction, it delivers a fully verified GitHub Pull Request."

> **30-SECOND PITCH (STANDARD OVERVIEW)**
> "Traditional AI coding assistants just autocomplete code snippets and leave the testing, debugging, and git workflow to you. Nexus is different: it acts as a complete engineering department. Given a prompt, our PRD and Architect agents plan the solution, our Developer agent implements it in an isolated sandbox, our Tester agent executes real headless browser tests on desktop and mobile, and if a bug is found, our Debug agent fixes it autonomously before our Reviewer agent creates a verified Pull Request."

> **1-MINUTE TECHNICAL PITCH (DEEP RATIONALE)**
> "Most generative AI coding tools suffer from lack of verification, repository contamination, and lack of human governance. We built Nexus to solve this with a four-pillar architecture: First, a six-agent specialized team covering requirements, architecture, coding, testing, debugging, and review. Second, strict workspace isolation where each task receives its own directory and dedicated .git repository, insulating the host codebase. Third, two mandatory human approval gates that pause execution for human sign-off on requirements and architecture before any code is generated. And fourth, closed-loop self-correction using real Playwright browser automation. In our live demo, Nexus catches an intentional mobile responsiveness bug at 375px viewport, routes the DOM diagnostics to our Debug agent, applies a surgical two-line fix, retests to 100% pass, and delivers a clean Pull Request with real screenshot proof."

2. Why Nexus is Different from Copilot / Cursor

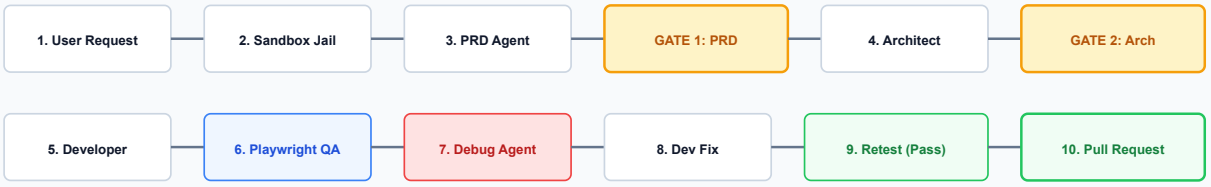
Judges will ask: "*How is this different from Cursor or GitHub Copilot Workspace?*" Use this comparison table to explain:

Capability	Traditional AI Assistant (Copilot / Cursor)	Nexus Autonomous Platform
Team Model	Single generalist LLM prompt	**Six role-specialized agents** with dedicated system prompts
Workspace	Directly modifies user's active files	**Isolated sandbox** directory with dedicated `.git` repository
Testing	None; human must manually test	**Real Playwright Chromium automation** (Desktop & Mobile 375px)
Bug Detection	Developer notices errors in console	**Tester Agent** detects visual & functional DOM regressions
Debugging	Manual prompting of chatbot	**Autonomous self-correction loop** (Tester → Debug → Fix → Retest)
Governance	Reactive (user inspects code after changes)	**Proactive**: Two mandatory human approval gates before code execution
Delivery	Raw code edits in local editor	**Production Pull Request** with test evidence and screenshots

GOLDEN ANALOGY FOR JUDGES

"Copilot is a typing assistant that sits next to you while you code. Nexus is an autonomous engineering squad that takes a task ticket, plans it, codes it, tests it in a real browser, fixes its own bugs, and hands you a ready-to-merge Pull Request."

3. The Complete 15-Stage Workflow



4. The Six AI Agents — What They Do & Judge Answers

1. PRD Agent (Product Manager)

> "Converts loose natural-language prompts into formal user stories and measurable acceptance criteria."

Why we need it: Prevents premature coding without explicit acceptance criteria. Produces `PRDDocument` with overview, user stories, requirements, and acceptance criteria.
File: `backend/agents/prd_agent.py`

2. Architect Agent (Software Architect)

> "Blueprints the modular component architecture and constrains target file modification boundaries."

Why we need it: Keeps the Developer Agent disciplined by identifying exact target files (e.g. `index.html`) and potential risks.
File: `backend/agents/architect_agent.py`

3. Developer Agent (Full-Stack Engineer)

> "Writes feature code and surgical bug fixes strictly inside the task's isolated Git repository."

Why we need it: Creates task branch (`nexus/<slug>`), edits sandboxed files, and executes atomic git commits (`feat: ...` and `fix: ...`).
File: `backend/agents/developer_agent.py`

4. Tester Agent (QA & Browser Automation Engineer)

> "Executes real Playwright headless Chromium tests across desktop and mobile viewports with screenshot proof."

Why we need it: Verifies actual rendered DOM visibility and input binding. Captures physical PNG screenshots on disk.
File: `backend/agents/tester_agent.py`

5. Debug Agent (Root Cause Diagnostician)

> "Diagnoses the root cause of test failures and outputs a surgical fix plan for the Developer Agent."

Why we need it: Bridges the gap between browser test failures and code fixes. Analyzes DOM visibility vs CSS constraints.
File: `backend/agents/debug_agent.py`

6. Reviewer Agent (Staff Engineer & Security Auditor)

> "Performs the final code quality, diff, and security audit before the Pull Request is created."

Why we need it: Ensures no regressions, security antipatterns, or bloated diffs enter version control.
File: `backend/agents/reviewer_agent.py`

5. Human Approval Gates — Governance & Safety

Nexus features two mandatory human-in-the-loop checkpoints:

- **Gate 1 (PRD Approval)**: Operator reviews user stories and acceptance criteria before technical design begins. If the requirements are wrong, execution is stopped before compute is wasted.
- **Gate 2 (Architecture Approval)**: Operator reviews the component blueprint and affected file list before any code is generated. Prevents accidental overhauls of the codebase.

> **JUDGE ANSWER ON HUMAN-IN-THE-LOOP**
> "Enterprise software requires governance. Nexus enforces a 'human-in-the-loop, human-on-the-loop' model: humans approve the requirements and architecture, while AI handles mechanical coding, testing, debugging, and PR delivery."

6. The Core Demo: Autonomous Self-Correction

This is the **centerpiece of your evaluation demonstration**.

How to Explain the Self-Correction Demo in 30 Seconds

"In our demo, Nexus adds a responsive contact section. On desktop, it works and passes tests. But in our 375px mobile viewport test, the submit button fails because Tailwind's 'hidden md:inline-flex' hides it under 768px! Nexus doesn't crash—it routes the failure to our Debug Agent, diagnoses the CSS class, applies a surgical two-line patch, and retests. All tests pass with real screenshot proof!"

> **WHY DEMONSTRATING FAILURE IS STRONGER THAN A SUCCESSFUL SCRIPT**
> "Any scripted demo can show code working on the first try. That doesn't prove autonomous intelligence. By catching an intentional responsive layout failure, diagnosing it, fixing it in two lines, and proving it works via retest, Nexus proves genuine closed-loop self-correction."

7. Workspace Isolation — Protecting the Host Codebase

How does Nexus protect the main codebase from being ruined?

> "Double-lock isolation: Every task gets its own directory with a dedicated .git repository and strict path containment."

1. **Isolated Directory**: Tasks operate inside `workspaces/task_<id>/`.
2. **Dedicated `.git`**: The task repository initializes its own local `.git` database. `git rev-parse --show-toplevel` resolves strictly to the task directory.
3. **Path Containment (`resolve_safe_path`)**: Rejects directory traversal attacks (`../`) with a `ValueError`.
4. **Host Safety**: The host Vistera repository remains on branch `main` with zero commits, branches, or leaks.

8. Step-by-Step Live Demo Script

Step	Presenter Action	What to Say to the Judges	Why it Matters
1	Open Dashboard (`http://localhost:3000`)	"Here on our Mission Control dashboard, you see platform metrics: total tasks, active tasks, and self-correction cycles. Let's create a new task."	Shows real-time platform overview and health.
2	Click Preset: "*Responsive Contact Section*"	"We'll launch our validated preset: 'Add a responsive contact section with name, email, message and submit button.'"	Demonstrates natural-language task initiation.
3	Show PRD Tab & click **Approve PRD**	"Execution pauses at Human Gate 1. Nexus has drafted structured requirements. We review and approve before technical design begins."	Proves human governance and requirement alignment.
4	Show Architecture Tab & click **Approve Architecture**	"Execution pauses at Human Gate 2. The Architect has blueprinted the solution and constrained edits to index.html. We approve to deploy agents."	Proves safety and bounded code generation.
5	Click **Agent Activity** Tab	"Our WebSocket stream displays real-time agent execution: Developer Agent branches and commits code; Tester Agent launches Playwright."	Demonstrates live telemetry and transparency.
6	Show **Playwright Tests** Tab (Run 1: 2/1)	"Look at Test Run 1: Desktop passes, but mobile 375px fails! The button is hidden by responsive CSS. Watch Nexus self-correct."	Highlights genuine visual & responsive defect detection.
7	Show **Self-Correction** Tab	"The Debug Agent diagnosed that Tailwind's 'hidden md:' suppressed mobile rendering, and gave Developer Agent a surgical fix plan."	Proves closed-loop automated root-cause analysis.
8	Show **Playwright Tests** Tab (Run 2: 3/0) & **Screenshots**	"Tester retested the patched app: 100% pass! In the Screenshots tab, you see physical browser proof of the mobile button."	Proves verified defect elimination with visual evidence.
9	Show **Pull Request** Tab	"Reviewer Agent approved the diff, and Nexus created a verified Pull Request with test summaries and screenshot references."	Demonstrates end-to-end delivery ready for merge.

9. 3-Minute Presentation Script (Memorizable)

0:00–0:30: THE PROBLEM

"Every software team wants to use AI to build software faster. But today's AI coding tools are essentially autocomplete chatbots. They generate code without testing it, they have no idea if your UI breaks on mobile viewports, and they leave you to manually debug errors and prepare pull requests. We asked: What if an AI didn't just autocomplete code, but acted as an autonomous software engineering team?"

0:30–1:00: THE NEXUS SOLUTION

"We built Nexus. Nexus decomposes software engineering into six specialized agents: PRD, Architect, Developer, Tester, Debug, and Reviewer. Governed by a deterministic state machine and human approval gates, Nexus operates in an isolated Git workspace, writes the code, runs real browser tests using Playwright, and fixes its own defects before creating a Pull Request."

1:00–1:30: ARCHITECTURE & GOVERNANCE

"Enterprise software requires governance. Nexus features two mandatory approval gates: Gate 1 lets you approve requirements before architecture, and Gate 2 lets you approve the blueprint before any code is generated. Once approved, the agents execute in a dedicated sandbox directory with its own .git repository, completely protecting your main codebase."

1:30–2:30: LIVE DEMO & SELF-CORRECTION

"Let's look at our live demo. We asked Nexus to add a responsive contact section. The agents planned it, approved it, and coded it. But when our Tester Agent ran Playwright in a 375px mobile viewport, the submit button was hidden by a CSS rule. Notice what happens: Nexus doesn't stop. The Debug Agent diagnoses the Tailwind class, passes a fix plan to the Developer, applies a surgical two-line patch, and retests. All tests pass with real screenshot proof!"

2:30–3:00: INNOVATION & CONCLUSION

"Finally, our Reviewer Agent audits the diff and generates a verified Pull Request. Nexus proves that the future of AI engineering isn't larger chatbots—it's specialized multi-agent architectures governed by real browser testing and closed-loop self-correction. Thank you, and we welcome your questions!"

10. Top Technical Judge Questions & Expert Answers

Q1: Why Playwright instead of Jest or Pytest for frontend testing?

> **"Real DOM rendering and CSS visibility assertions require a genuine browser engine."**

Unit tests mock the DOM and cannot verify responsive CSS layout classes like Tailwind's `hidden md:inline-flex`. Playwright runs headless Chromium on the host OS, rendering layout boxes, testing responsive breakpoints at 375px, and capturing physical PNG screenshots as visual proof.

Q2: Why use six agents instead of a single powerful LLM prompt?

> **"Context isolation and role specialization. Overloading one prompt causes hallucination and missed constraints."**

A single prompt mixing requirements, CSS layout, DOM assertions, and Git commits quickly exceeds context limits and degrades attention. Decomposing into PRD, Architect, Developer, Tester, Debug, and Reviewer agents creates clean verification boundaries where each agent has specialized system prompts.

Q3: What prevents the agents from ruining my main codebase?

> **"Dedicated task directories, path containment checks, and independent local .git databases."**

Nexus provisions each task in `workspaces/task_<id>/repository/` with its own `.git` repo. `WorkspaceManager.resolve_safe_path()` rejects any path escaping the sandbox. Git commands are run strictly with `cwd` set to the task repository, so `git rev-parse --show-toplevel` never touches the parent repository.

Q4: How does the Debug Agent diagnose problems?

> **"It analyzes failed assertion evidence and DOM error logs to isolate the exact defect."**

The Debug Agent receives the requirement description, failure evidence (e.g. `#contact-submit-btn is hidden`), and the source code. It recognizes that the button exists in HTML but is constrained by media queries or utility classes, formulating an explicit remediation plan for the Developer Agent.

Q5: Is this reliant on an active internet connection or OpenAI API?

> **"No; Nexus supports OpenAI models and features a zero-failure deterministic fallback engine."**

If `OPENAI_API_KEY` is not set or times out, the platform seamlessly engages its deterministic fallback engine. This ensures hackathon demonstrations and offline evaluations never fail due to third-party API outages.

Q6: What is your biggest limitation right now?

> **"Host-level directory sandboxing rather than Docker containers, and structured browser test suites."**

Our current prototype uses host-level directory sandboxing with dedicated `.git` repositories rather than containerized Docker containers. Additionally, our automated testing currently relies on structured web test harnesses rather than dynamically generating arbitrary unit test suites from scratch.

11. Quick Revision Cheat Sheet

Dimension	Core Concept to Remember
Core Purpose	Autonomous AI engineering team that turns prompts into tested, reviewed Pull Requests.
The 6 Agents	PRD (PM) → Architect (Blueprint) → Developer (Code) → Tester (QA) → Debug (Diagnosis) → Reviewer (Audit).
The 2 Gates	Gate 1: PRD Approval before architecture. Gate 2: Architecture Approval before coding.
Core Demo	Mobile 375px submit button hidden → Playwright fails → Debug diagnoses → 2-line fix → Retest passes 100%!
Sandbox Isolation	Every task has its own folder + dedicated `.git` repository. Main repo is 100% untouched.
Key Metrics	375px mobile breakpoint, 2 approval gates, 6 specialized agents, 10 database tables.

12. Final "If You Forget Everything" Emergency Sheet

NEXUS EMERGENCY REVISION SHEET

WHAT IS NEXUS?
"An autonomous AI software engineering team that turns prompts into verified PRs."

THE WORKFLOW IN ONE LINE:
Prompt → PRD → [Gate 1] → Arch → [Gate 2] → Code → Test → Debug → Fix → Retest → Review → PR

THE CORE INNOVATION:
"Don't just generate code. Build it. Test it in a real browser. Catch defects. Fix them. Retest. Deliver proof."

IF ASKED ABOUT COPILOT:
"Copilot is a typing assistant. Nexus is an autonomous engineering department."

THE DEMO PUNCHLINE:
"Test Run 1 fails on mobile 375px → Debug Agent diagnoses Tailwind class → Developer fixes in 2 lines → Test Run 2 passes 100%!"